Lecture 4: Classification foundations, Generative models

Max G'Sell

Machine Learning II: 46-927

February 11, 2019

Announcements

- Homework 3 is due today. Homework 4 comes out tomorrow night and is due on February 18.
- I will be lecturing from NYC next week (February 18).

Today

Last time we switched from regression to classification. Today we will continue on classification ideas and introduce generative models

- Remarks about the homework
- Assessing classifiers
- Generative models
  - Example: Linear Discriminant Analysis
  - Example: Naive Bayes *(maybe)*

Where we were

We switched from regression to classification.

Now we observe pairs $(X, Y) \sim \mathcal{P}$, with $X \in \mathbb{R}^p$ and $Y \in \{0, 1\}$ (or sometimes $\{-1, 1\}$).

Based on n observed pairs, we estimate a function $\hat{f}$ that "guesses Y well" for future $(X, Y) \sim \mathcal{P}$.

In a classification problem, we can make $K(K-1)$ different kinds of mistakes! This leads to two different $K \times K$ matrices reflecting classification performance.

Confusion matrix and loss

Confusion Matrix, which tabulates counts of what actually happened.

	True $Y = 1$	True $Y = 0$
Guess $Y = 1$	True Positive	False Positive
Guess $Y = 0$	False Negative	True Negative

Loss Matrix, which tabulates how bad each outcome combination is for you.

	True $Y = 1$	True $Y = 0$
Guess $Y = 1$	0	L_{01}
Guess $Y = 0$	L_{10}	0

Confusion matrix: error rates

	True $Y = 1$	True $Y = 0$
Guess $Y = 1$	True Positive	False Positive
Guess $Y = 0$	False Negative	True Negative

We are interested in many different quantities in the confusion matrix.
The most famous is the misclassification rate: $(FN + FP)/n$

If False positives and False negatives are equally bad, then minimizing the misclassification rate is a good goal.

If we set our loss to have $L_{01} = L_{10} = 1$ and $L_{00} = L_{11} = 0$, minimizing the expected loss will be the same as minimizing the expected misclassification rate.

Statistical decision theory

We showed that the expected misclassification error (or expected prediction error),

$$\mathrm{EPE} = \mathbb{E}\left(\mathcal{L}(Y, \hat{f}(X))\right) = \mathbb{E}\left(\mathbf{1}_{Y \neq f(X)}\right),$$

is minimized by the ideal rule:

$$f(x) \underset{g \in \{1,\ldots,K\}}{\operatorname{argmax}} \mathbb{P}(Y = g | X = x),$$

if we knew $\mathbb{P}(Y = g|X = x)$ for all g. This is called the Bayes Classifier, and the error rate it achieves is the Bayes Rate.

Building a classifier

Combining this with Bayes' theorem leads to two formulations of an ideal classifier:

$$\begin{aligned}\hat{f}(x) &= \underset{j=1,\ldots K}{\operatorname{argmax}}\ \mathrm{P}(Y = j|X = x) \\ &= \underset{j=1,\ldots K}{\operatorname{argmax}}\ \mathrm{P}(X = x|Y = j) \cdot \pi_j\end{aligned}$$

1. **Discriminative models:** We can try to estimate $\mathbb{P}(Y = j|X = x)$ directly. (Discriminative models)

2. **Generative models:** We could try to estimate the distributions $\mathbb{P}(X = x|Y = j)$ for each class.

Logistic regression

We introduced logistic regression as a first example of a discriminative classifier.

What we are doing: Directly estimating a minimizer of $\mathbb{E}\left(\mathbf{1}_{Y \neq f(X)}\right)$ over functions $f(x)$ (even linear functions) is *hard*.

We take a simpler approach: estimating $\mathbb{P}(Y = k|X = x)$, and then plugging into the ideal classification rule.

(A similar idea arises all over, including our upcoming discussion of SVM and Boosting.)

Logistic regression

We take a simpler approach: estimating $\mathbb{P}(Y = k|X = x)$, and then plugging into the ideal classification rule.

In logistic regression, we choose a simple, familiar form for a parametric model of $\mathbb{P}(Y = k|X = x)$:

$$\log\left\{\frac{\mathrm{P}(Y = 1|X = x)}{\mathrm{P}(Y = 0|X = x)}\right\} = \beta^T x$$

or equivalently,

$$\mathrm{P}(Y = 1|X = x) = \frac{e^{\beta^T x}}{1 + e^{\beta^T x}} = \frac{1}{1 + e^{-\beta^T x}}$$

Estimating logistic regression

It is much easier to estimate $P(Y = k|X = x)$; we just use maximum likelihood!

Consider the model of Y_i:

$$Y_i \sim \text{Bernoulli}(p(x_i))$$

$$p(x_i) = \frac{e^{\beta^T x}}{1 + e^{\beta^T x}}$$

We just write out the likelihood for this model in terms of β and optimize:

$$\hat{\beta} = \underset{\beta \in \mathbb{R}, \beta \in \mathbb{R}^p}{\text{argmax}} \sum_{i=1}^{n} \left\{ y_i \cdot (\beta^T x_i) - \log\left(1 + \exp(\beta^T x_i)\right) \right\}$$

Classification by logistic regression

Once we have $\hat{\beta}$, we can just plug in to our ideal decision rule to get a classifier.

$$\begin{aligned}\hat{f}(x) &= \underset{k}{\operatorname{argmax}}\, \mathbb{P}(Y = k | X = x) \\ &= \begin{cases} 1 & \text{if } \frac{\exp(x^T\hat{\beta})}{1+\exp(x^T\hat{\beta})} > 0.5 \\ 0 & \text{otherwise} \end{cases}\end{aligned}$$

And if we invert the monotonic transformation, this is equivalent to:

$$\hat{f}(x) = \begin{cases} 1 & \text{if } x^T\hat{\beta} > 0 \\ 0 & \text{otherwise} \end{cases}$$

Remark: Outputs from logistic regression

Extension: Multinomial Logistic Regression

We can generalize logistic regression to K classes, leveraging the same ideas.

We now have *vectors* $\beta_1, \ldots, \beta_K$, and define

$$\mathbb{P}(Y = k|X = x) = \frac{e^{x_i^T \beta_k}}{\sum_{j=1}^{K} e^{x_i^T \beta_j}}$$

It turns out that the β_k are not uniquely identifiable, you can eliminate one of them.

These probabilities are given by the *softmax* function, which we will see again in neural nets.

Other extensions of logistic regression

We saw that we can:

1. Add a ridge penalty to our optimization to reduce variance and allow solutions when $p > n$, just like linear regression.

2. Add a lasso penalty to our optimization to give sparsity, reduce variance, and allow solutions when $p > n$, just like linear regression.

3. Use an additive model instead of a linear model to allow more flexible fits, just like linear regression:

$$\log\left(\frac{P(Y=1|X=x)}{P(Y=0|X=x)}\right) = \sum_{j=1}^{p} \hat{f}_j(x)$$

Classification so far

Focus on binary classification,

- $Y = 1$ if an event of interest happened
- $Y = 0$ if it did not happen

Most of our classifiers will give us a guess $\hat{p}(x) = \hat{\mathbb{P}}(Y = 1|X = x)$.
This is an estimate of the true $p(x) = \mathbb{P}(Y = 1|X = x)$.

We saw that to minimize average misclassification error,
$\mathbb{E}\mathbf{1}_{f(X)\neq Y}$, you should choose $f(x) = \mathbf{1}_{p(x)>1/2}$.

Let's think about this more carefully.

Confusion matrix

Back to classifying

- $Y = 1$ if an event of interest happened
- $Y = 0$ if it did not happen

Recall: In K-class classification, we can make $K(K-1)$ different kinds of mistakes!

We will be thinking more about these confusion matrices.

In our 2×2 setting:

	True $Y = 1$	True $Y = 0$
Guess $Y = 1$	True Positive	False Positive
Guess $Y = 0$	False Negative	True Negative

Misclassification rate

Suppose that I tell you my classifier has 97% accuracy. Is this good?

It depends! What if I am predicting lottery wins. The probability of winning may be $1/175000000$.

In that case, guessing "no" has a misclassification rate of 5×10^{-9}...

We need to consider the *base rate*: the probability of each class given no further information.

Confusion matrix: Example

For the homework problem, and the classification rule $\hat{p}(x) > 0.5$, our confusion matrix was something like

	True $Y = 1$	True $Y = 0$
Guess $Y = 1$	0	2
Guess $Y = 0$	1083	7957

Issues with imbalanced data and misclassification rates

The idea of minimizing the misclassification rate is tempting — no one wants to be wrong!

However, this is often not the real goal of applying a classifier. Losses are usually asymmetric! Sometimes you even care directly about $\hat{p}(x)$!

Furthermore, in settings where the classes are imbalanced, the misclassification rate is very difficult to improve and is often misleading.

Imbalanced data

In many common applications, our classes are imbalanced:

- Marketing
- Fraud detection
- Mortgage default
- Whether there will be a large jump in price after an event.

Misclassification rate and imbalanced data

We know that we should classify as $Y = 1$ when $P(Y = 1|X = x) > 0.5$ to minimize expected misclassification rate. Suppose that an event only happens 5% of the time in the general population.

By Bayes Theorem,

$$P(Y = 1|X = x) =$$

Therefore, to classify as $Y = 1$, we would need to see an X that is more likely in the $Y = 1$ population than the general population.

Even if there is useful information in the data, it is probably not enough to flip our classifications!

Confusion matrix: error rates

	True $Y = 1$	True $Y = 0$
Guess $Y = 1$	True Positive	False Positive
Guess $Y = 0$	False Negative	True Negative

Other error rate measures better capture asymmetric concerns when classifying:

- Sensitivity (Recall, Hit rate):

- Specificity (True Negative Rate, TNR):

These quantities will allow us to better think about cost trade offs.

Sensitivity and Specificity

Sensitivity: Fraction of points with $Y = 1$ that we find.

- Want high Sensitivity when False Negatives are more costly than False Positives. Example: Fraud detection.

Specificity: Fraction of points with $Y = 0$ that we avoid.

- Want high Specificity when False Positives are more expensive than False Negatives. Example: Criminal trials.

We would like both of them to be high, but we need to make tradeoffs as we adjust our threshold.

- We can flag more cases as fraud to improve sensitivity, but this will decrease our specificity.

Cost/Loss for classification

	True $Y = 1$	True $Y = 0$
Guess $Y = 1$	0	L_{01}
Guess $Y = 0$	L_{10}	0

You showed on your homework that the loss is minimized if you guess $Y = 1$ when

$$\mathbb{P}(Y = 1|X = x) >$$

If it costs \$5 dollars to call someone who will not buy, but it costs \$100 to miss someone who would buy, you should

This suggests that we can trade off costs and error rates by just changing the threshold on $\mathbb{P}(Y = 1|X = x)$!

Changing threshold on $\mathbb{P}(Y = 1|X = x)$

As we change the threshold on $\mathbb{P}(Y = 1|X = x)$, we vary the size and quality of the set we reject.

We can think about $\mathbb{P}(Y = 1|X = x)$ as a function for ranking our observations by likelihood of $Y = 1$.

It is convenient to look at the Sensitivity and Specificity of our classifier as the threshold changes. This corresponds to the ROC (Receiver operating characteristic) curve that you plotted in your homework.

Aside: Classification scores

So far, we've been talking about classification based on estimates $\hat{p}(x) = \hat{\mathbb{P}}(Y = 1|X = x)$.

In fact, we don't really need probabilities to make classifications. A more general concept is a classification *score*, $\hat{f}(x)$, such that $\hat{f}(x)$ is big for

Any threshold on $\hat{f}(x)$ will yield a confusion matrix, which in turn yields all of the error metrics we have discussed, as well as ROC curves.

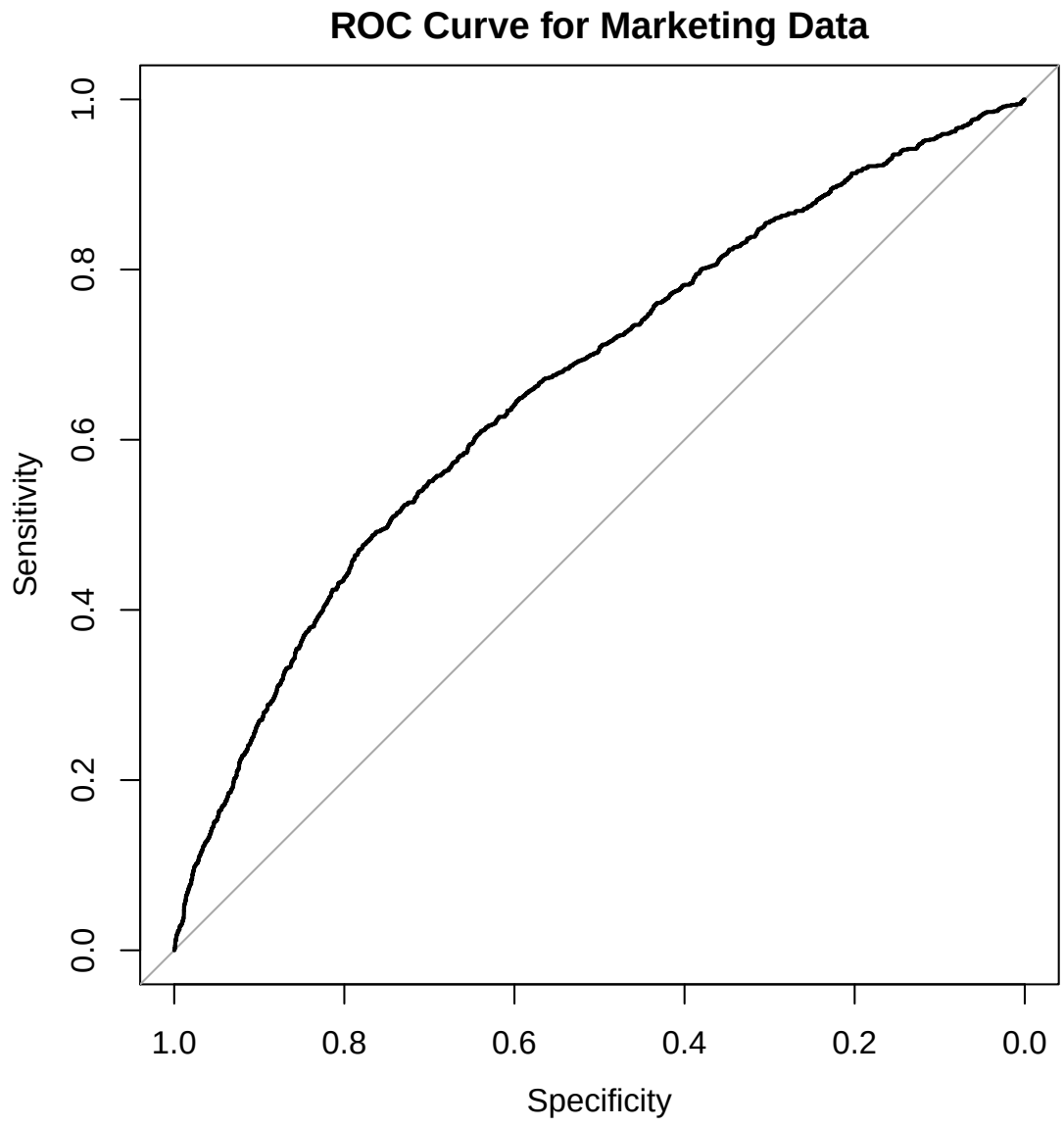
- Question: How do ROC curves change if I monotonically warp my classification score?

Aside: Classification scores

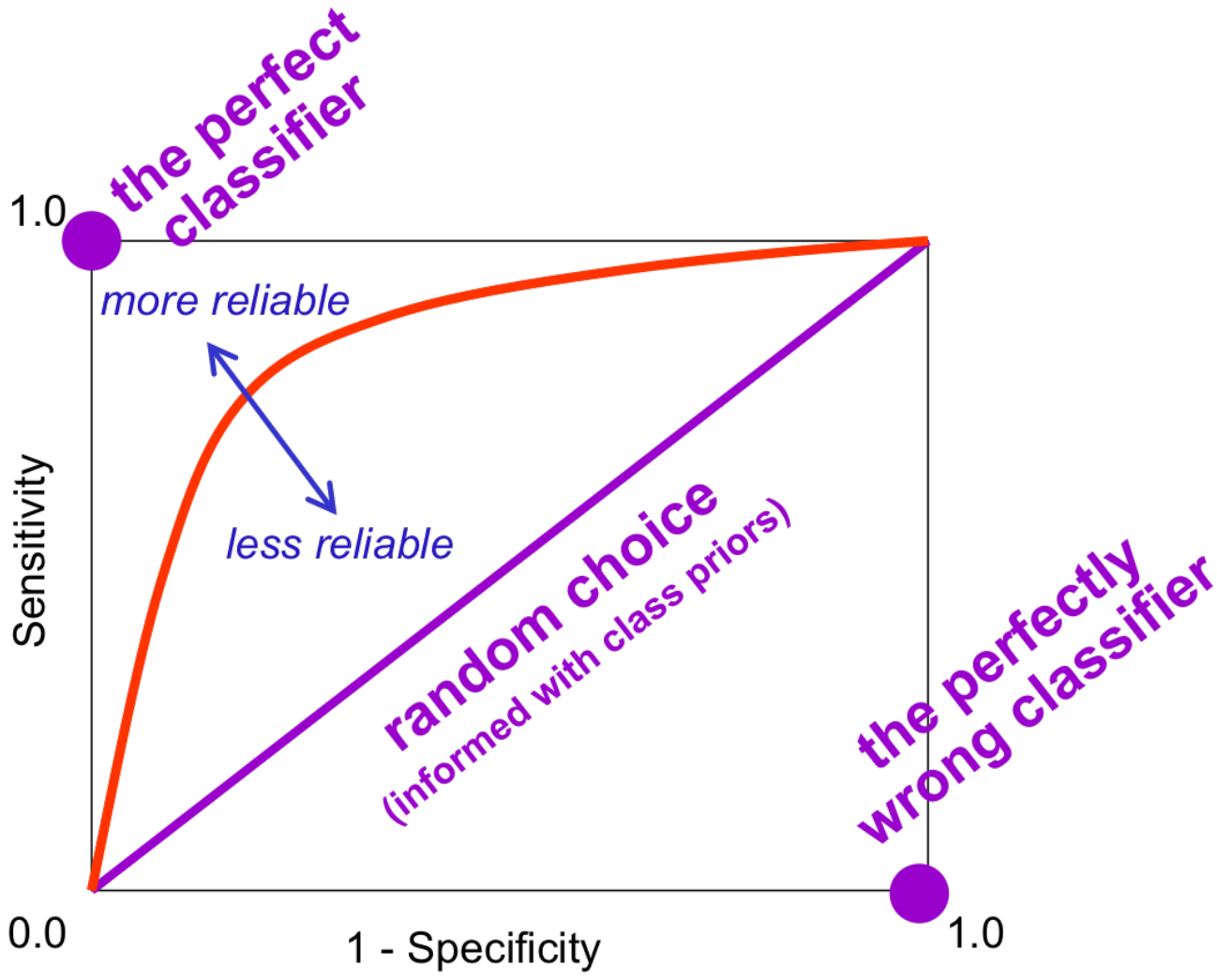
Examples of classification scores that are not probabilities:

It is often useful to think of the distribution of $\hat{f}(x)|Y = k$ for understanding classifier behavior. (Homework)

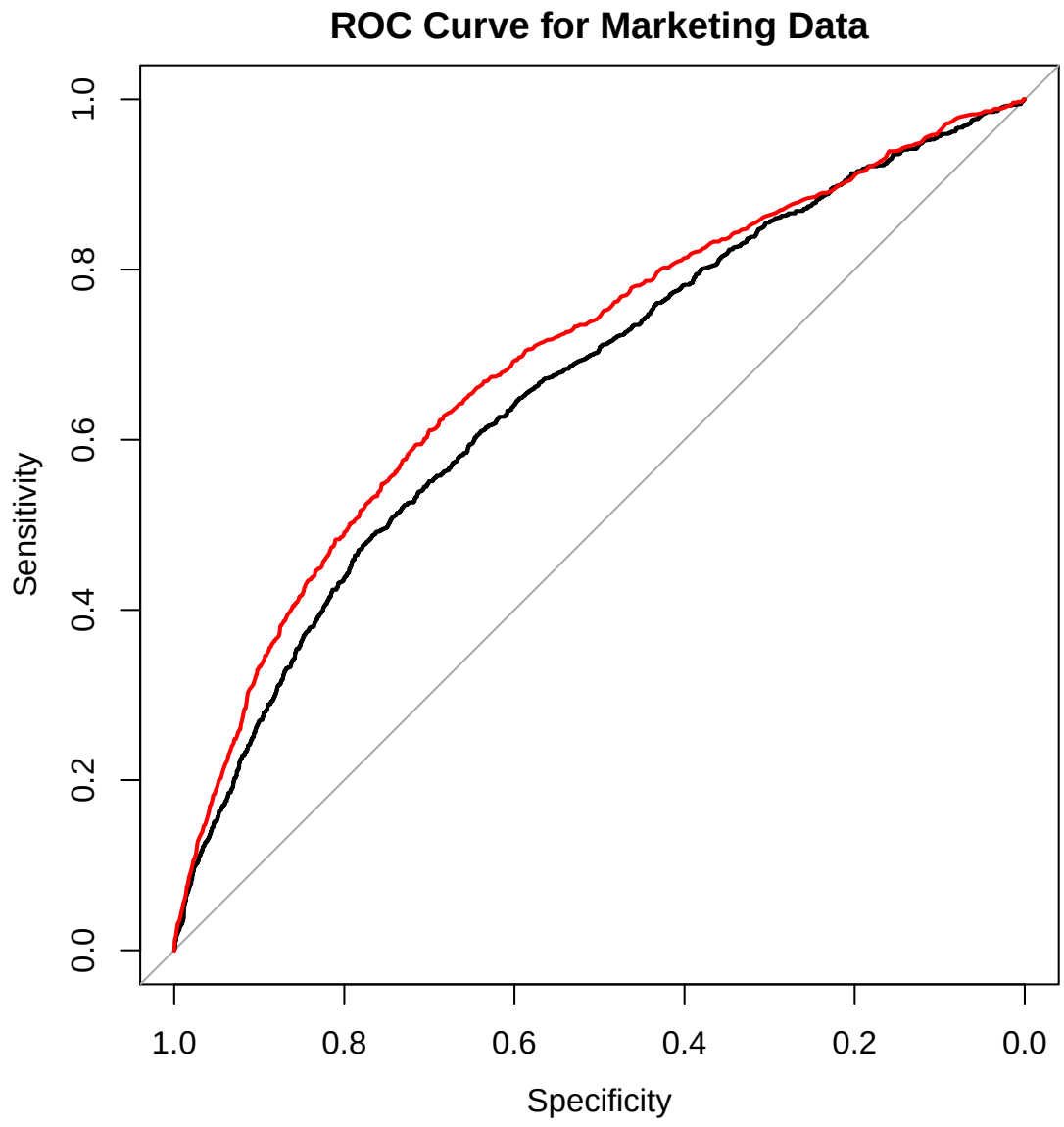
ROC Curve


ROC Curve for Marketing Data
Sensitivity
0.0
0.2
0.4
0.6
0.8
1.0
1.0
0.8
0.6
0.4
0.2
0.0
Specificity

ROC Curve


the perfect classifier
1.0
more reliable
less reliable
Sensitivity
random choice
(informed with class priors)
the perfectly wrong classifier
0.0
1 - Specificity
1.0

ROC Curve (again)



ROC Curve

A generally higher curve tends to be better, though you should keep tradeoffs in mind.

The Area Under the Curve (AUC) is one way to measure this. A higher area tends to be more attractive.

AUC has a nice interpretation:

If you randomly pick two observations from your distribution and order them by $\hat{p}(x)$, the AUC is the probability that your ordering is correct!

In reality though, you are interested in the performance cutoff at a particular point on the curve

Other metrics

Positive predictive value (PPV):

$$PPV = \frac{\#\text{ observations correctly classified as positive}}{\#\text{observations classified as positive}}$$
$$= \frac{TP}{TP + FP}$$

Suppose that you take a test for a disease. The PPV captures the probability that you actually have the disease when you test positive.

Other metrics

Negative Predictive Value (NPV):

$$NPV = \frac{\#\text{ observations correctly classified as negative}}{\#\text{observations classified as negative}}$$
$$= \frac{TN}{TN + FN}$$

Suppose that you take a test for a disease. The NPV is the probability that you are not sick, given that the test comes up negative.

Calibration

Is our $\hat{p}(x)$ a good estimate of $p(x) = \mathbb{P}(Y = 1|X = x)$?

Why do we care? We're just going to threshold it...

There is information in knowing $\mathbb{P}(Y = 1|X = x)$.

- A case that is definitely fraud ($p(x) = 0.99$) may be handled very differently than a case that might be fraud ($p(x) = 0.52$).

  Calculating the proper trade-offs for making decisions depends on the probabilities, not just the categories.

Note: Even if $\hat{p}(x)$ is not a good approximation of $p(x)$, thresholding $\hat{p}(x)$ might still give good guess at categories.

Calibration plots

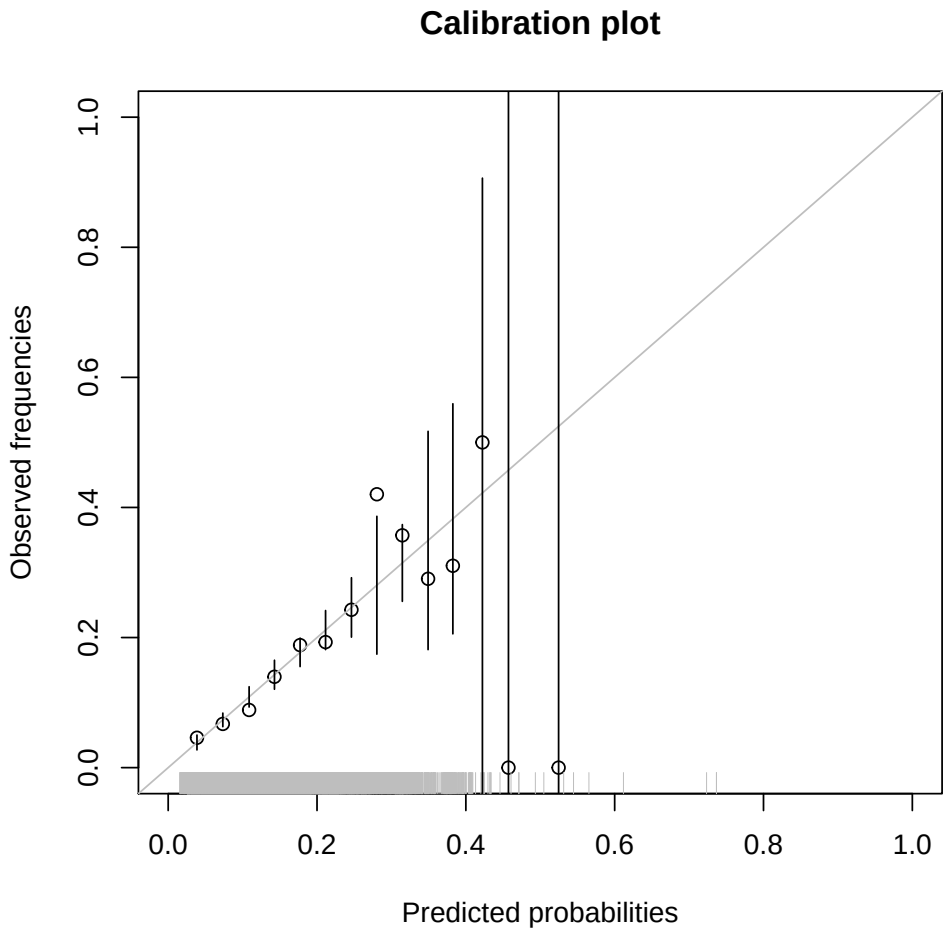
To check how well calibrated your $\hat{p}(x)$ is, you can make a *calibration plot*:

1. Bin the data according to predicted probability, $\hat{p}(x)$. E.g., $[0, 0.1], (0.1, 0.2], \ldots, (0.9, 1]$.

2. For each bin, calculate the proportion of observations with class $Y = 1$

3. Plot these true fractions against the midpoints of the bin predicted probabilities

Calibration plots



Calibration plots

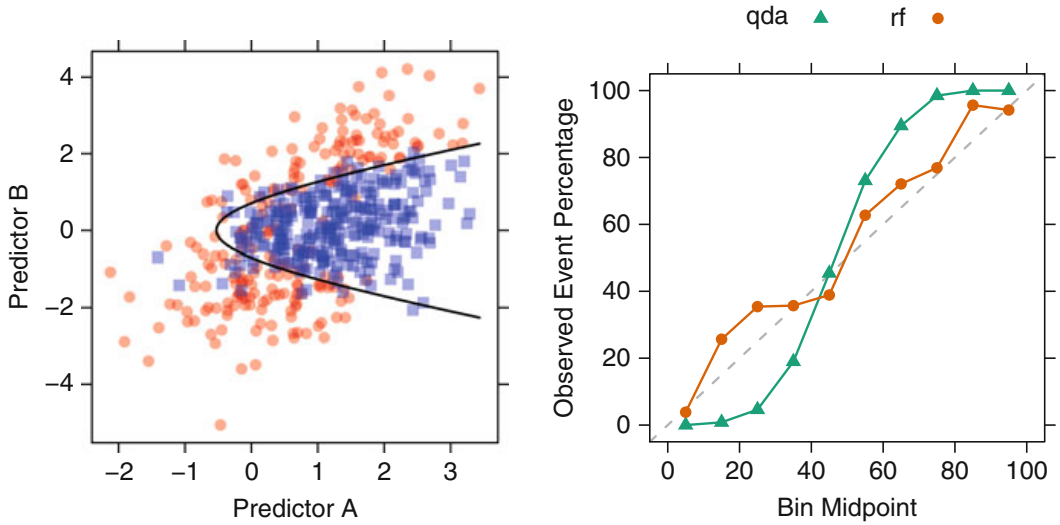


Fig. 11.1: *Left*: A simulated two-class data set with two predictors. The *solid black line* denotes the 50 % probability contour. *Right*: A calibration plot of the test set probabilities for random forest and quadratic discriminant analysis models

(From *Applied Predictive Modeling* by Kjell Johnson and Max Kuhn)

If the calibration is off, you can try transforming your $\hat{p}(x)$.

Remark: Model validation and tuning

Suppose that we want to pick between different classification models. Or instead, that we need to choose λ in logistic lasso. How can we do this?

Where we are now

We have the basic idea of classification, and we've seen one sort of classifier.

We need to see several more. The big classics are:

- Simple generative models: LDA, Naive Bayes.
- Support Vector Machines
- Trees, Random forests, Boosting

It's worth spending some time on each of these. Then we'll know enough to talk about general modeling/tuning advice, followed by unsupervised learning and advanced topics (ML2)

Switching topics: Building a classifier

We showed that the ideal classifier is the Bayes classifier:

$$\begin{aligned}\hat{f}(x) &= \underset{k=1,\ldots K}{\operatorname{argmax}} \ \mathrm{P}(Y = k|X = x) \\ &= \underset{k=1,\ldots K}{\operatorname{argmax}} \ \mathrm{P}(X = x|Y = k) \cdot \pi_k\end{aligned}$$

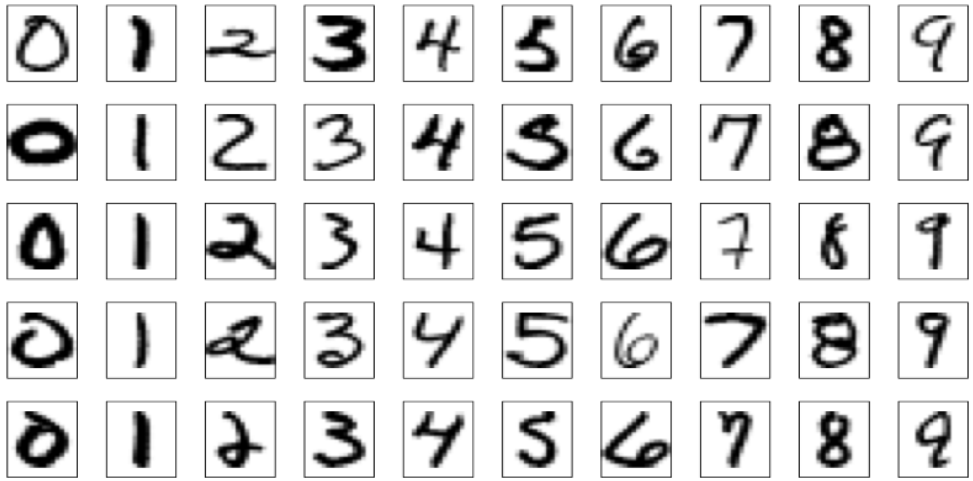
To build a classifier, we either:

1. Estimate $\mathbb{P}(Y = k|X = x)$ directly. (Discriminative models)
2. **Estimate both π_k and $\mathbb{P}(X = x|Y = k)$ for each class. (Generative models)**

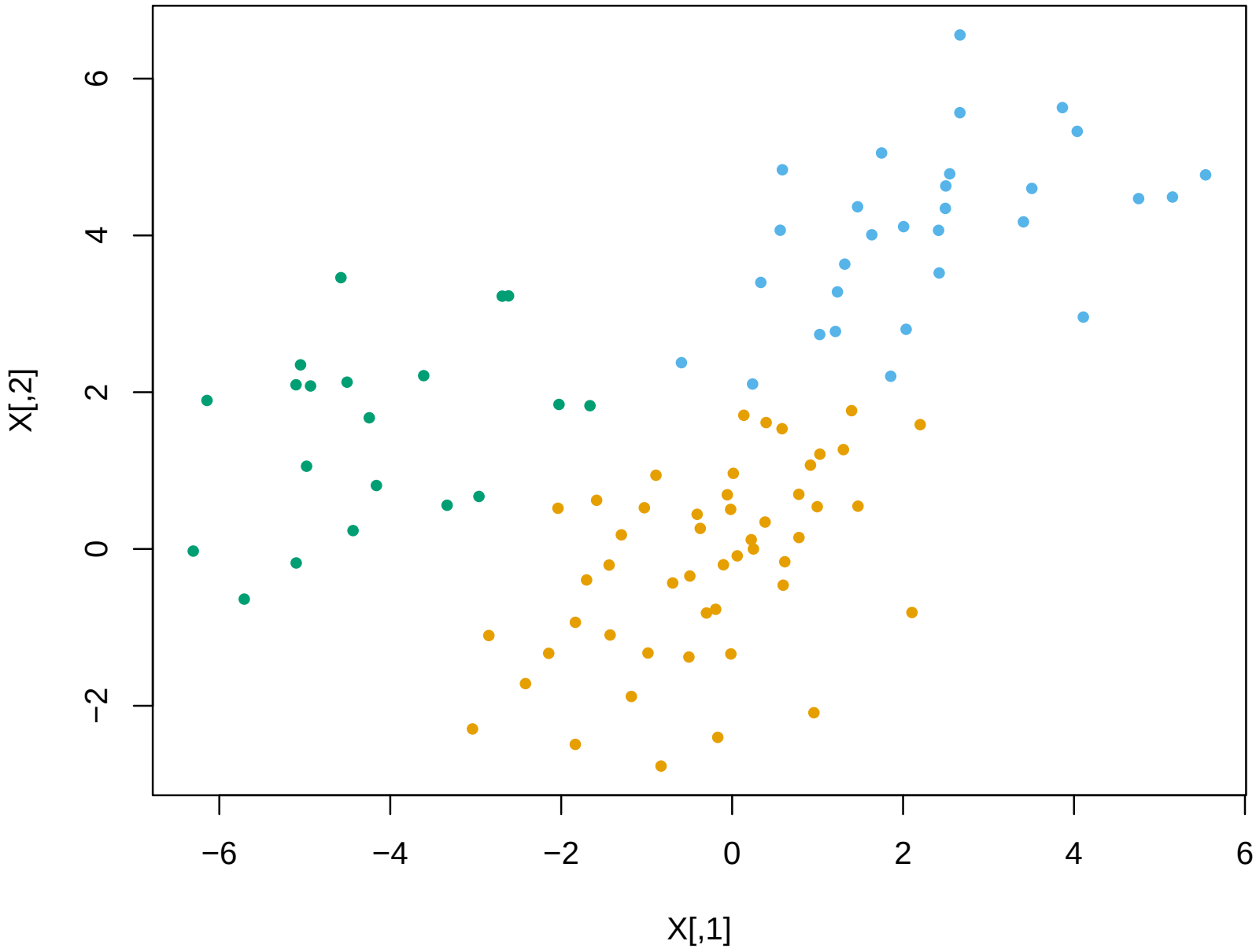
Generative models

Generative models are nice, because they let us think about modeling the process that *generated the data*

Think about the classic MNIST digits data:



It might be easier to describe what a 4 looks like than how all the digits are different.


X[,2]
6
4
2
0
−2
−6
−4
−2
0
2
4
6
X[,1]

Generative models

To implement a generative classifier, we need to decide how to model and estimate π_k and $\mathbb{P}(X = x|Y = k)$. π_k is easy, the distribution of $X|Y = k$ is the main question.

A simple, famous choice is to model the distribution of $X|Y = k$ as multivariate Gaussian.

We will look at this briefly, partly as a demonstration of generative models, and partly because it will give us an excuse to introduce several more general useful ideas.

Sidenote: Generative models and the joint distribution

Linear discriminant analysis

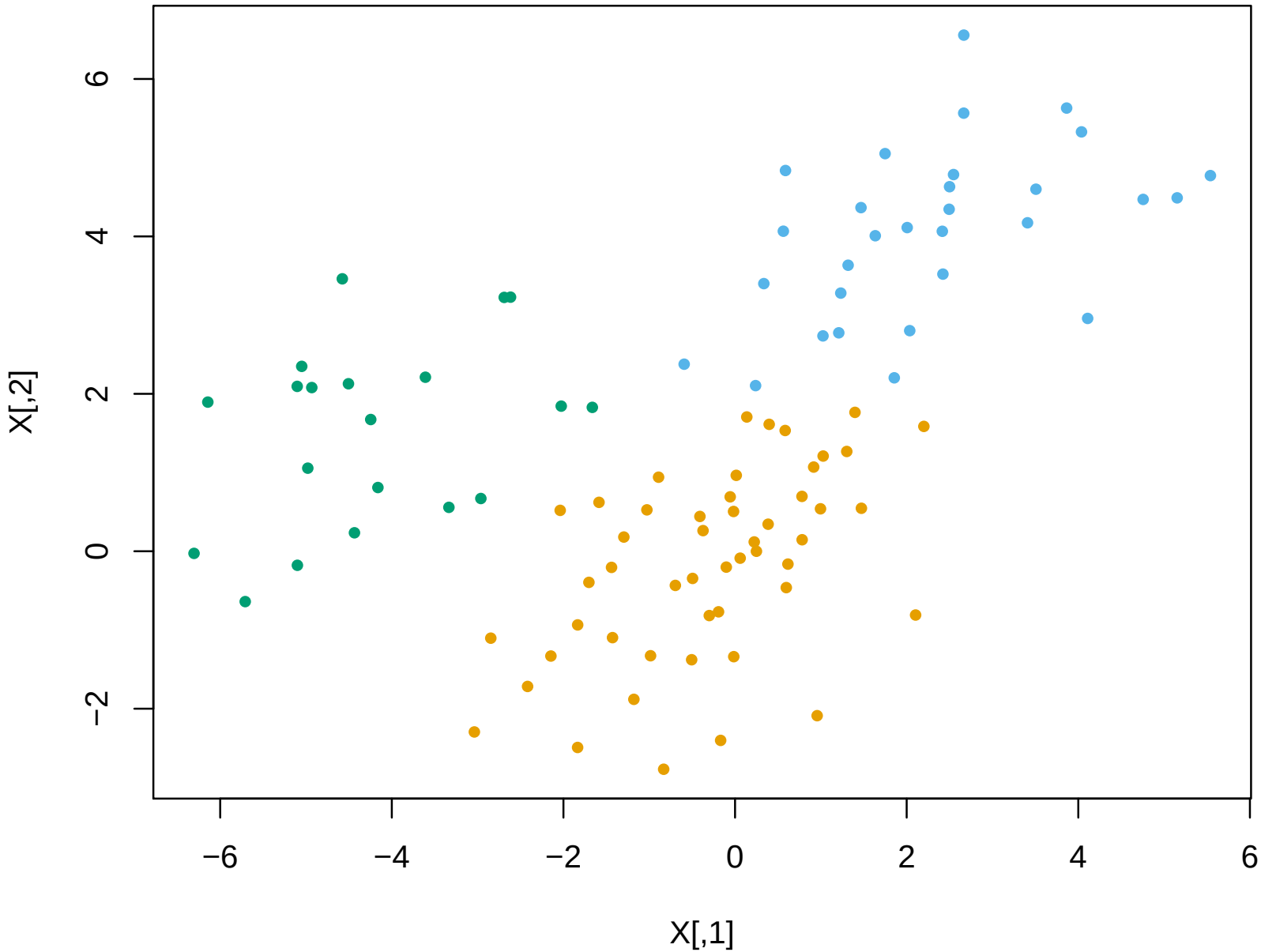
Linear discriminant analysis (LDA) models the data X within each class as being normally distributed:

$$h_j(x) = \mathrm{P}(X = x | Y = j) = N(\mu_j, \Sigma) \text{ density}$$

We allow each class to have its own mean $\mu_j \in \mathbb{R}^p$, but we assume a common covariance matrix $\Sigma \in \mathbb{R}^{p \times p}$. Hence

$$h_j(x) = \frac{1}{(2\pi)^{p/2} \det(\Sigma)^{1/2}} \exp\Big\{ -\frac{1}{2}(x - \mu_j)^T \Sigma^{-1} (x - \mu_j) \Big\}$$

Think of this as a model for classification problems where the different classes look like shifted clumps.


X[,2]
6
4
2
0
−2
−6
−4
−2
0
2
4
6
X[,1]

Linear discriminant analysis

What does the decision rule look like? We want to find j so that $\mathrm{P}(Y = j|X = x) \cdot \pi_j = h_j(x) \cdot \pi_j$ is the largest. We can define the rule:

$$
\begin{aligned}
f^{\mathrm{LDA}}(x) &= \underset{j=1,\ldots K}{\operatorname{argmax}} \ \frac{1}{(2\pi)^{p/2}\det(\Sigma)^{1/2}} e^{-\frac{1}{2}(x-\mu_j)^T\Sigma^{-1}(x-\mu_j)} \cdot \pi_j \\
&= \underset{j=1,\ldots K}{\operatorname{argmax}} \\
&= \underset{j=1,\ldots K}{\operatorname{argmax}} \\
&= \underset{j=1,\ldots K}{\operatorname{argmax}} \ \delta_j(x)
\end{aligned}
$$

We call $\delta_j(x)$, $j = 1, \ldots K$ the discriminant functions. Note

$$
\delta_j(x) = x^T\Sigma^{-1}\mu_j - \frac{1}{2}\mu_j^T\Sigma^{-1}\mu_j + \log \pi_j
$$

is just an affine function of x

LDA: Estimation

In practice, we have to estimate the parameters π_j, μ_j, and Σ. We estimate them based on the training data $x_i \in \mathbb{R}^p$ and $y_i \in \{1, \dots K\}$, $i = 1, \dots n$, by:

- $\hat{\pi}_j = n_j/n$, the proportion of observations in class j
- $\hat{\mu}_j = \frac{1}{n_j} \sum_{y_i = j} x_i$, the centroid of class j
- $\hat{\Sigma} = \frac{1}{n-K} \sum_{j=1}^{K} \sum_{y_i = j} (x_i - \hat{\mu}_j)(x_i - \hat{\mu}_j)^T$, the pooled sample covariance matrix

(Here n_j is the number of points in class j)

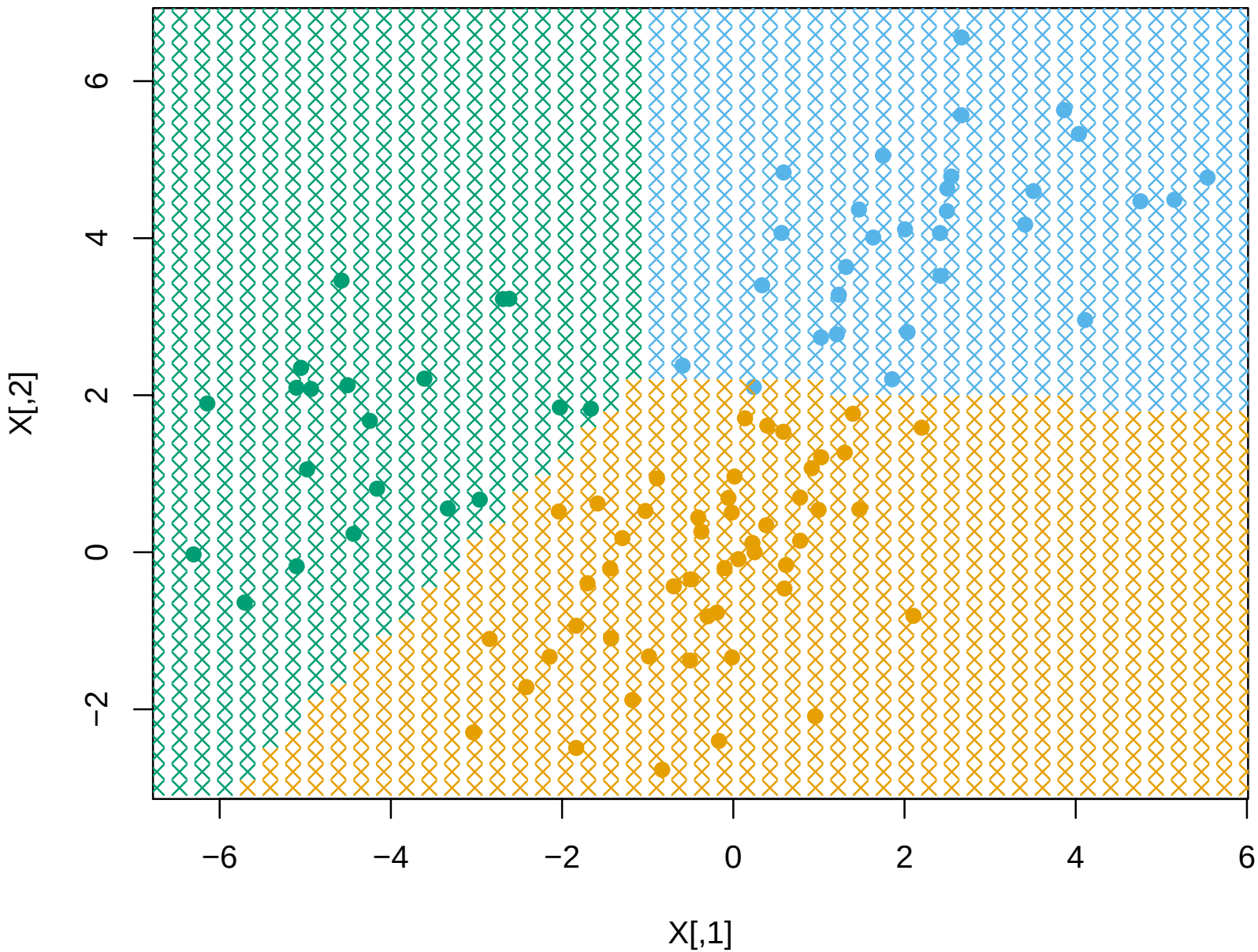
LDA: Estimation

This gives the estimated discriminant functions:

$$\hat{\delta}_j(x) = x^T \hat{\Sigma}^{-1} \hat{\mu}_j - \frac{1}{2} \hat{\mu}_j^T \hat{\Sigma}^{-1} \hat{\mu}_j + \log \hat{\pi}_j$$

and finally the linear discriminant analysis rule for new $x \in \mathbb{R}^p$,

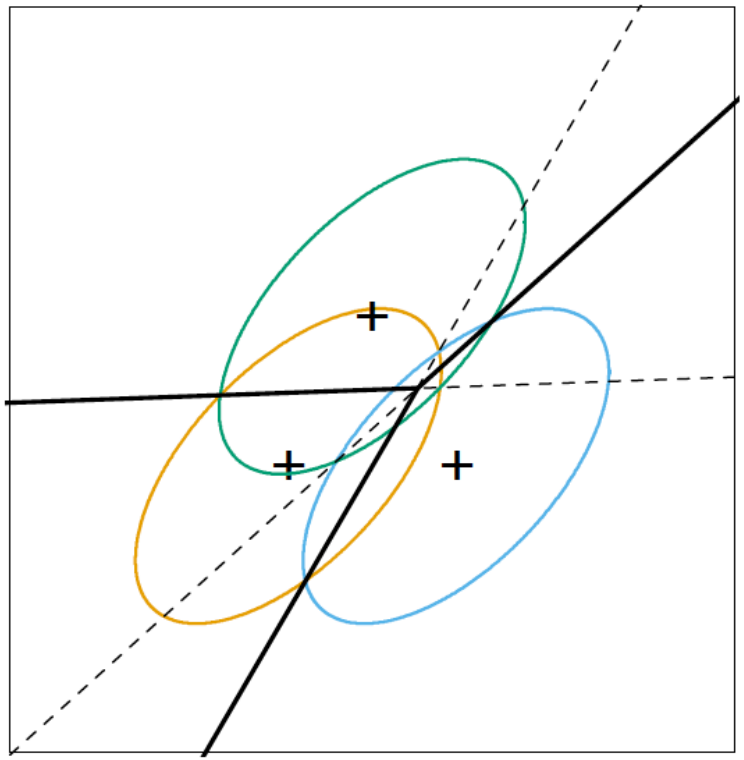
$$\hat{f}^{\text{LDA}}(x) = \underset{j=1,\ldots K}{\text{argmax}} \ \hat{\delta}_j(x)$$

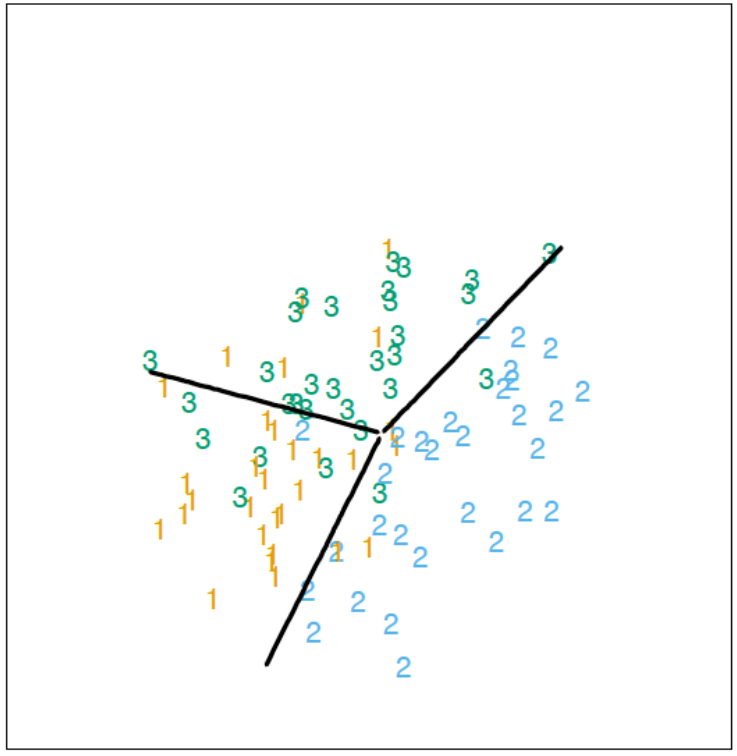

X[,2]
6
4
2
0
−2
−6
−4
−2
0
2
4
6
X[,1]

Example: LDA decision boundaries

Example of decision boundaries from LDA (from ESL page 109):

$f^{\mathrm{LDA}}(x)$ $\hat{f}^{\mathrm{LDA}}(x)$





Are the decision boundaries the same as the perpendicular bisectors between the class centroids?

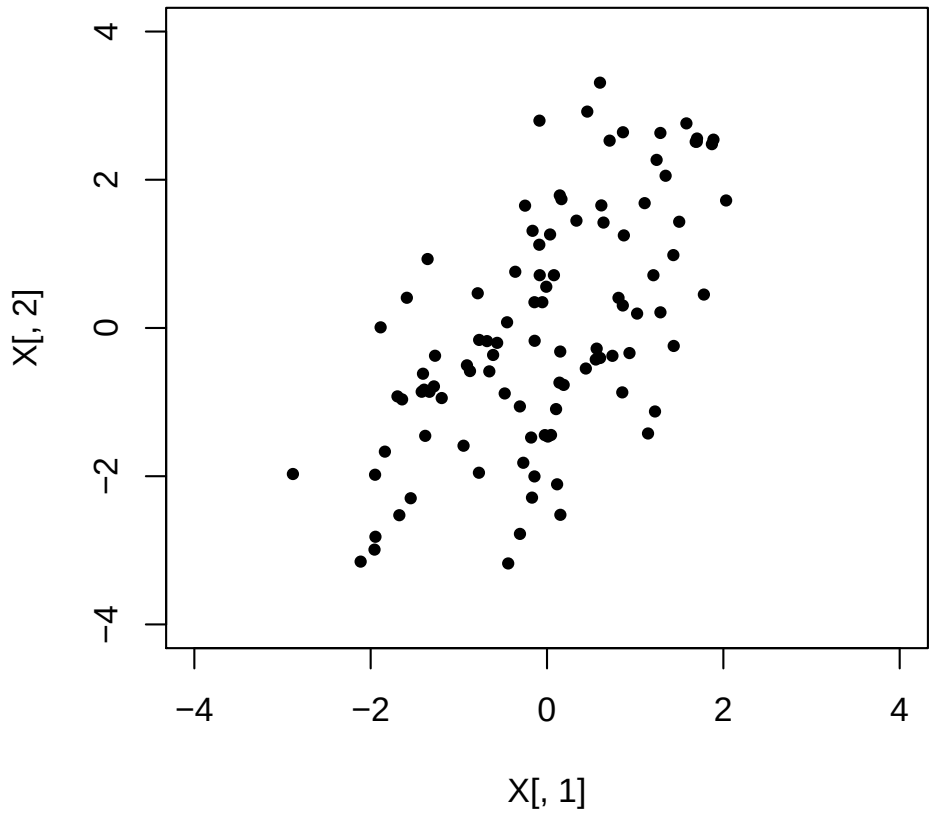
Thinking about the decision function

$$\hat{f}^{\mathrm{LDA}}(x) = \underset{j=1,\ldots K}{\operatorname{argmax}}\, x^T \hat{\Sigma}^{-1} \hat{\mu}_j - \frac{1}{2} \hat{\mu}_j^T \hat{\Sigma}^{-1} \hat{\mu}_j + \log \hat{\pi}_j$$

What changes if our loss function is not 0-1? What changes in the population proportion π_k changes?

Thinking about the decision function

Let's look at the $(x_i - \mu_k)^T \Sigma^{-1} (x_i - \mu_k)$ term that appears in the LDA decison rule. How should we think about this?



Mahalanobis distance

Mahalanobis distance measures the distance from a center in terms of the variance of the distribution.

For a Gaussian, it captures how far out in the tail of the distribution a point is.

This is used for LDA, QDA, and Multivariate Gaussian distributions.

It can also be used for outlier detection and for clustering!

Suppose I've seen a lot of data, and now a new point comes along. How can I tell whether it's a "rare" or outlier point?

Mahalanobis distance, PCA, and LDA

Note that LDA equivalently minimizes over $k = 1, \ldots K$,

$$\frac{1}{2}(x - \hat{\mu}_k)^T \hat{\Sigma}^{-1}(x - \hat{\mu}_k) - \log \hat{\pi}_k$$

It helps to factorize $\hat{\Sigma}$ (i.e., compute its eigendecomposition):

$$\hat{\Sigma} = UDU^T$$

where $U \in \mathbb{R}^{p \times p}$ has orthonormal columns (and rows), and $D = \mathrm{diag}(d_1, \ldots d_p)$ with $d_k \geq 0$ for each k. Then we have $\hat{\Sigma}^{-1} = UD^{-1}U^T$, and

$$(x - \hat{\mu}_k)^T \hat{\Sigma}^{-1}(x - \hat{\mu}_k) =$$

This is just the squared distance between $\tilde{x}$ and $\tilde{\mu}_k$!

LDA transformed

Hence the LDA procedure can be described as:

1. Compute the sample estimates $\hat{\pi}_k, \hat{\mu}_k, \hat{\Sigma}$
2. Factor $\hat{\Sigma}$, as in $\hat{\Sigma} = UDU^T$
3. Transform the class centroids $\tilde{\mu}_k = D^{-1/2}U^T\hat{\mu}_k$
4. Given any point $x \in \mathbb{R}^p$, transform to $\tilde{x} = D^{-1/2}U^T x \in \mathbb{R}^p$, and then classify according to the nearest centroid in the transformed space, adjusting for class proportions—this is the class k for which $\frac{1}{2}\|\tilde{x} - \tilde{\mu}_k\|_2^2 - \log \hat{\pi}_k$ is smallest

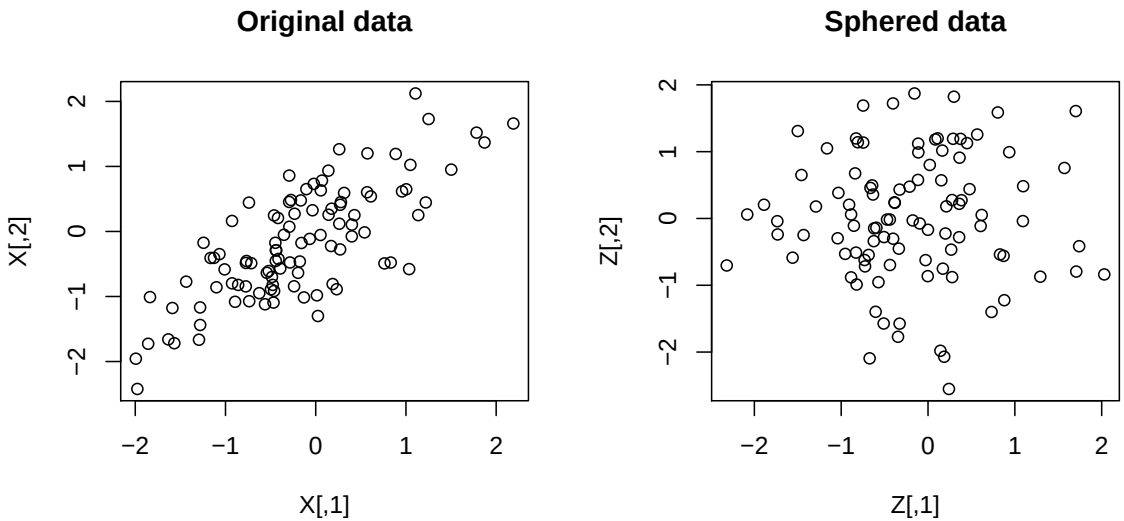
Sphering

What is this transformation doing? Think about applying it to the observations:

$$\tilde{x}_i = D^{-1/2} U^T x_i, \quad i = 1, \ldots n$$

This is basically sphering the data points, because if we think of $x \in \mathbb{R}^p$ were a random variable with covariance matrix $\hat{\Sigma}$, then

$$\mathrm{Cov}(D^{-1/2} U^T x) =$$



Linear subspace spanned by sphered centroids

LDA compares the quantity $\frac{1}{2}\|\tilde{x} - \tilde{\mu}_k\|_2^2 - \log \hat{\pi}_k$ across the classes $k = 1, \ldots K$.
Think about the $K - 1$ dimensional plane that goes through all K centroids, $\tilde{\mu}_1, \ldots, \tilde{\mu}_K$.

Because we are looking at distance to these centroids, only the distance projected onto this plane matters!

LDA procedure summarized

We can think of the LDA procedure as:

1. Compute the sample estimates $\hat{\pi}_k, \hat{\mu}_k, \hat{\Sigma}$

2. Make two transformations: first, sphere the data points, based on factoring $\hat{\Sigma}$; second, project down to the affine subspace spanned by the sphered centroids. This can all be summarized a single linear transformation $A \in \mathbb{R}^{(K-1)\times p}$

3. Given any point $x \in \mathbb{R}^p$, transform to $\tilde{x} = Ax \in \mathbb{R}^{K-1}$, and classify according to the class $k = 1, \ldots K$ for which

$$\frac{1}{2}\|\tilde{x} - \tilde{\mu}_k\|_2^2 - \log \hat{\pi}_k$$

is smallest, where $\tilde{\mu}_k = A\hat{\mu}_k$

This way of describing LDA may sound more complicated, but actually, it's much simpler! After applying A, we've reduced the problem from p to $K-1$ dimensions, and then it's basically nearest centroid classification:

$$\hat{f}^{\mathrm{LDA}}(x) = \underset{k=1,\ldots K}{\operatorname{argmin}} \ \frac{1}{2}\|\tilde{x} - \tilde{\mu}_k\|_2^2 - \log \hat{\pi}_k$$

(The only distinction being that we adjust for class proportions)

How do these linear classifiers compare?

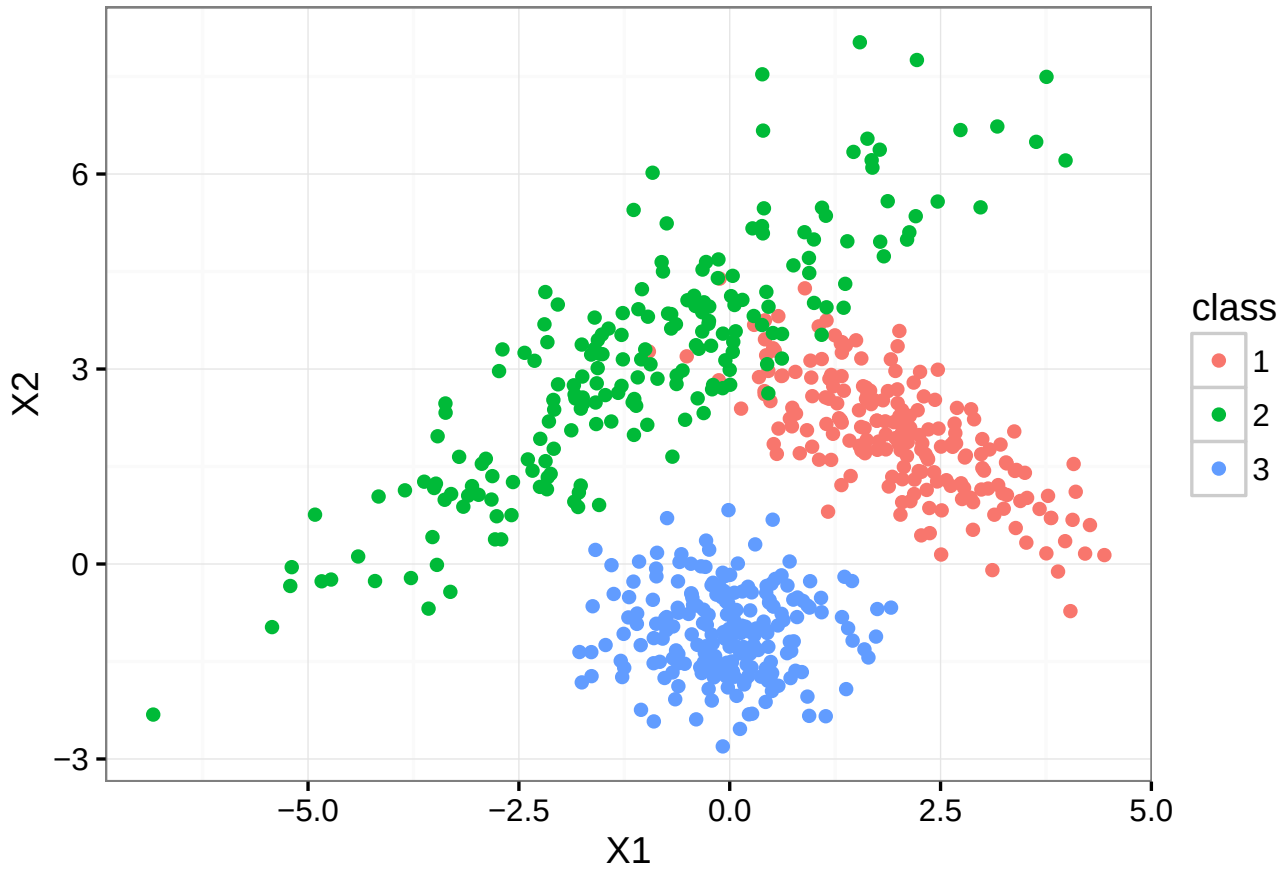
Though the decision rules are both linear, these methods produce different classifications because they rely on different assumptions.

1. LDA: Works well if the groups are in "clumps" so that the Gaussian distribution is reasonable. Also assumes the shapes are similar.

2. Logistic: Only requires that $\log \frac{P(Y=1|X=x)}{P(Y=0|X=x)} \approx x_i^T \beta$, which is strictly weaker. LDA will do better when it's assumptions are reasonable, but otherwise worse. Logistic focuses more on the boundary points.

Variations on LDA: unequal Σ

The LDA model assumes that our covariance is the same for all groups. What if it's not?



$$\Sigma_1 = \begin{pmatrix} 1 & -0.7 \\ -0.7 & 1 \end{pmatrix}, \Sigma_2 = \begin{pmatrix} 3 & 2.5 \\ 2.5 & 3 \end{pmatrix}, \Sigma_3 = \begin{pmatrix} 0.5 & 0 \\ 0 & 0.5 \end{pmatrix}$$

All the same math goes through, giving Quadratic Discriminant Functions (and QDA)

$$\begin{aligned}\delta_k(x) &= -\frac{1}{2}(x-\mu_k)^T\Sigma^{-1}(x-\mu_k) - \frac{1}{2}\log|\Sigma_k| + \log\pi_k \\ &= -\frac{1}{2}x^T\Sigma_k^{-1}x + \mu_k\Sigma_k^{-1}x - \frac{1}{2}\mu_k^T\Sigma_k^{-1}\mu_k - \frac{1}{2}\log|\Sigma_k| + \log\pi_k\end{aligned}$$

The Σ matrices are now different in each function are now different, and the boundaries $\delta_k(x) > \delta_{k'}$ are no longer linear. Instead, they are

This allows the boundaries to curve around groups to account for different patterns of spread. It comes at a cost though:

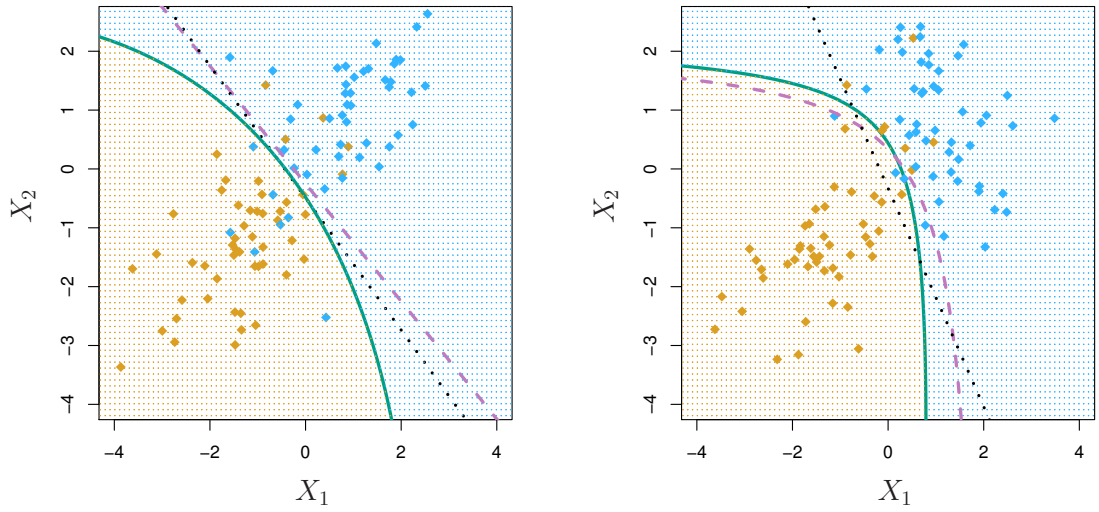
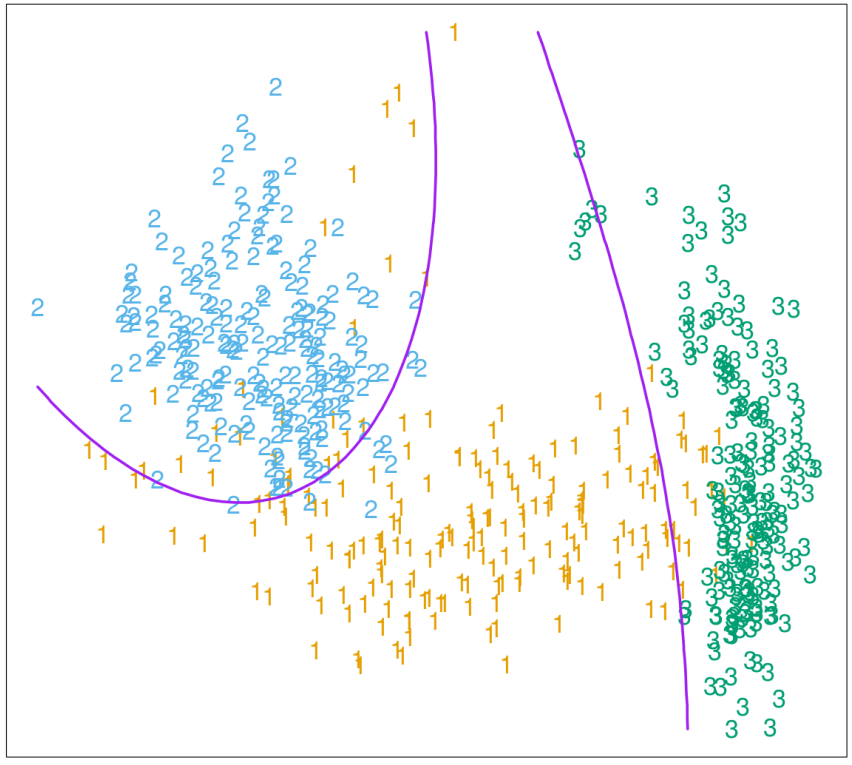


Figure 4.9 from ISL. Dashed purple curve is the Bayes classifier decision boundary. Solid green curve is QDA, dotted black line is LDA. Left: True boundary is linear. Right: True boundary is quadratic

Variations on LDA: unequal Σ

Quadratic discriminant analysis (QDA):



(ESL 4.6)

Variations on LDA: high dimensions

LDA really becomes instructive when we consider its performance over a range of dimensions.

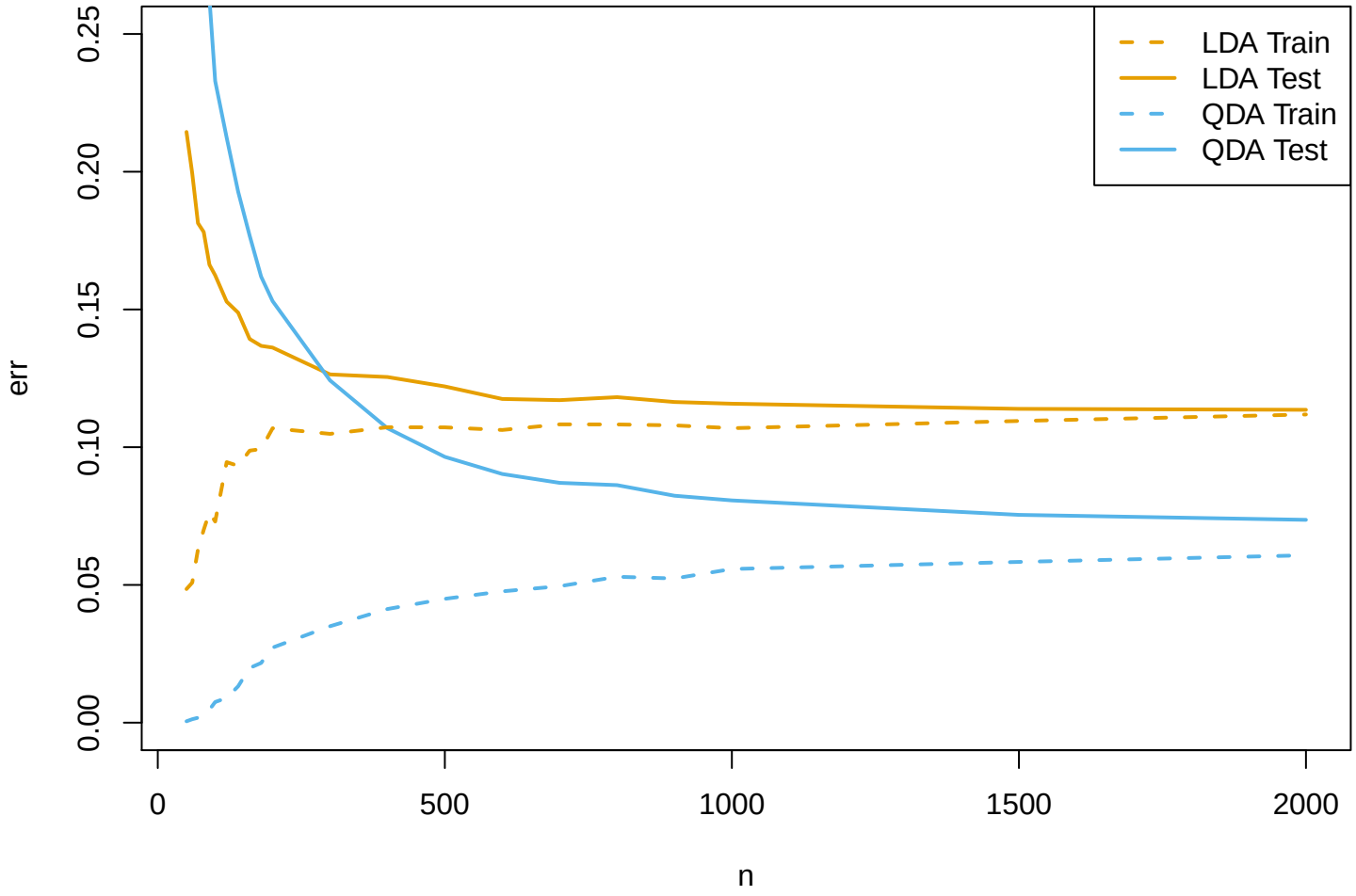
Fitting QDA requires estimating

Fitting LDA requires estimating

As the number of parameters increases, the of our estimator increases (but the hopefully decreases).

Suppose that we have two groups, drawn respectively from $N(\mu_1, \Sigma_1)$ and $N(\mu_2, \Sigma_2)$. If we choose to use LDA instead of QDA, we are choosing a more biased model to reduce variance.

Variations on LDA: high dimensions



Variations on LDA: high dimensions

Suppose the dimension gets even higher?

What if I can't even estimate the group means well?

Naive Bayes

Imagine that you have $n = 2000$ observations and $p = 1000$ features.

It will be incredibly hard to estimate $f_k = P(X = x|Y = k)$ well for any complicated model!

You need very strong "assumptions" on f_k (reducing parameters/variance)!

Naive Bayes *assumes* (well, models) that all of the components of $X = (X_1, \ldots, X_p)$ are *independent* (conditional on Y).

Naive Bayes

Naive Bayes models all of the components of $X = (X_1, \ldots, X_p)$ as *independent* (conditional on Y).

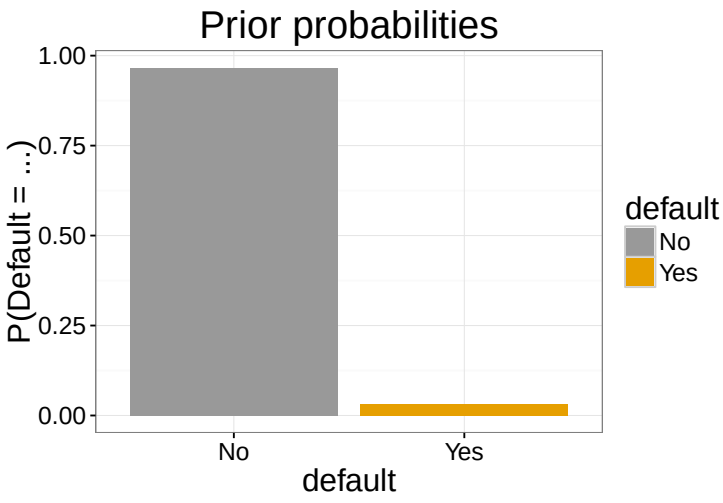
Under this independence assumption, the class distributions factor!

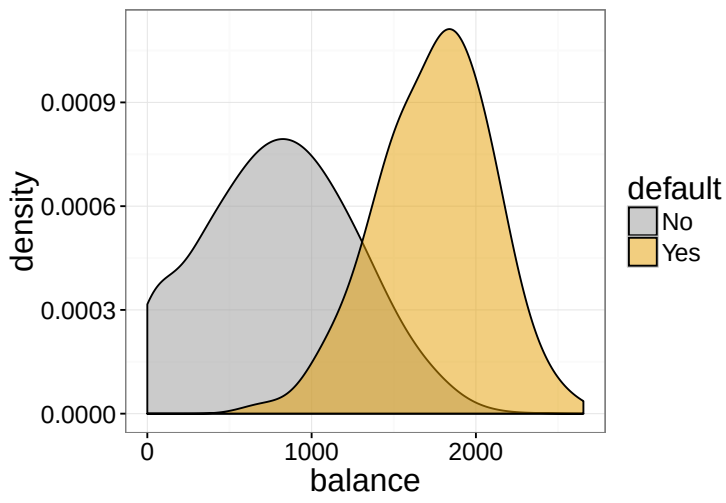
$$
\begin{aligned}
f_k(x) &= P(X = x | Y = k) \\
&= \\
&= \\
&=
\end{aligned}
$$

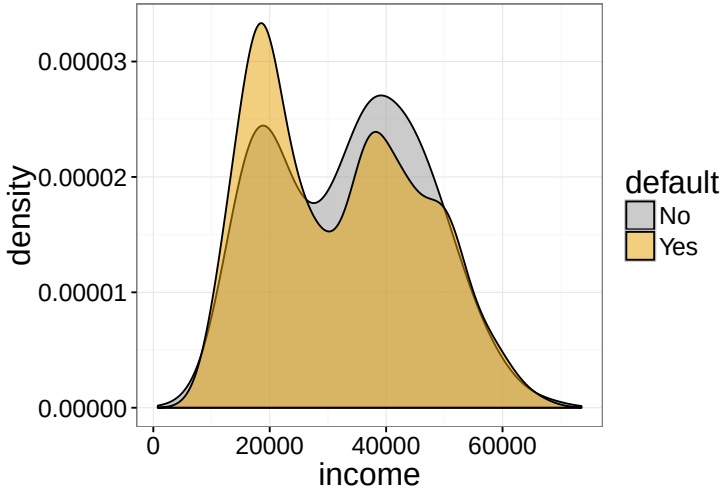
This means that we can just estimate the *univariate* distributions!

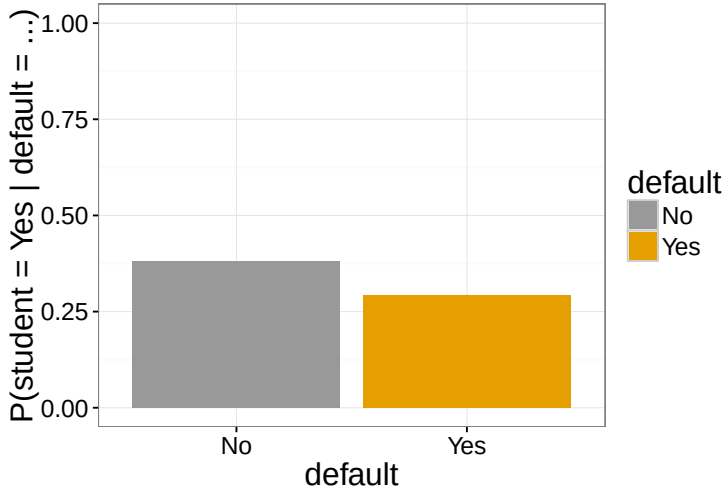
$$f_k^{(j)}(x_j) = P(X_j = x_j | Y = k)$$

Example: Default data from ISLR









We can easily calculate these simplified class distributions:

$$\hat{f}_{Yes} = \hat{f}_{Yes}(\text{income})\hat{f}_{Yes}(\text{balance})\hat{f}_{Yes}(\text{student})$$
$$\hat{f}_{No} = \hat{f}_{No}(\text{income})\hat{f}_{No}(\text{balance})\hat{f}_{No}(\text{student})$$

And then plug them into the Bayes classifier formula, just like we did for LDA

$$P(\text{default}|i, b, s) = \frac{\hat{\pi}_{Yes}\hat{f}_{Yes}(i, b, s)}{\hat{\pi}_{Yes}\hat{f}_{Yes}(i, b, s) + \hat{\pi}_{No}\hat{f}_{No}(i, b, s)}$$

Naive Bayes

Naive Bayes scales well to problems with very large p. We only need enough data to estimate each of the marginal distributions well.

It also allows you to have a flexible choice of models for each of the univariate distributions.

However, Naive Bayes cannot capture *interactions* between the features within each class!

LDA and QDA are able to incorporate these feature interactions, at the cost of needing to estimate them.